

项目

LexDraft AI 智能法律文书系统

基于大语言模型的司法文书咨询·草拟·修正·优化桌面系统，支持多种法律文书类型的智能生成与多轮法律咨询对话。

项目介绍

智能法律文书系统是一款面向个人用户的桌面端法律文书辅助工具，通过调用大语言模型 API，根据用户输入的案件材料自动生成规范的法律文书，并提供文书修正优化与智能法律顾问对话功能。

核心功能：

-  **文书智能生成** — 支持 17+ 种法律文书类型，涵盖法院诉讼类（民事/行政/刑事）和非诉讼诉求文书（举报信、检举信、仲裁申请等），每种文书配备专业提示词模板
-  **修正优化** — 对已生成的文书进行格式规范、用语严谨、逻辑审查等多维度专业优化
-  **智能法律顾问** — 多轮对话式法律咨询，以口语化方式提供法律分析、风险提示和行动建议
-  **类案推送与法条关联** — 文书生成后自动检索关联法律条文、相似参考案例与诉讼风险提示，右侧可折叠面板实时展示
-  **导出文件** — 支持将生成结果导出为 Word (.docx) 文档，自动处理中文字体与排版
-  **灵活配置** — 支持自定义 API 地址、模型、Temperature 等参数，兼容 OpenAI 格式的各类大模型接口

支持的文书类型：

分类	子类	文书
法院诉讼类	民事诉讼	民事起诉状、答辩状、保全申请书、民事上诉状、民事再审申请书
法院诉讼类	行政诉讼	行政起诉状、行政上诉状、行政再审申请书
法院诉讼类	刑事当事人可用文书	刑事自诉状、刑事附带民事起诉状、刑事申诉书
非诉讼诉求文书	行政机关举报	举报信
非诉讼诉求文书	纪检监察专属	检举信（纪委监委）、控告书（纪委监委）、处分复查纪检申诉书
非诉讼诉求文书	信访诉求材料	信访材料
非诉讼诉求文书	仲裁调解保全执行	仲裁申请书、调解申请书、诉前保全申请书、执行申请书

技术栈： Python 3.12 · Tkinter · python-docx · BeautifulSoup4 · markdown · PyInstaller

快速开始

面向使用者（使用 exe 文件）

1. 前往 [Releases](#) 下载最新版本的 智能法律文书服务系统.exe 及 config.ini
2. 将 exe 文件和 config.ini 放在同一目录下
3. 用文本编辑器打开 config.ini，填入你的 API 配置：

```
[AI]
URL = https://api.deepseek.com/v1/chat/completions
API_KEY = 你的API密钥
MODEL = deepseek-v4-flash
TEMPERATURE = 0.5
```

4. 双击运行 exe 文件
5. 首次使用请先在左侧菜单点击「系统设置」确认 API 配置正确

注意： config.ini 必须与 exe 文件位于同一目录，prompt 文件夹也应在同一目录下以加载提示词模板。

面向开发者（从源码运行）

```
# 克隆项目
git clone <项目地址>

# 进入项目目录
cd 智能法律文书系统

# 创建并激活虚拟环境
python -m venv venv
venv\Scripts\activate # windows
source venv/bin/activate # Linux/Mac

# 安装依赖
pip install -r requirements.txt

# 运行程序
python main.py
```

打包为 exe：

```
pip install pyinstaller
pyinstaller main.spec
```

打包产物位于 dist/ 目录，需将 config.ini 和 prompt/ 文件夹一同分发。

实现方法

RAG 提示词构建流程（离线阶段）

本系统的核心优势在于 `prompt/` 目录下的每份提示词都不是手工编写的通用模板，而是通过以下 RAG 流程精心生成：

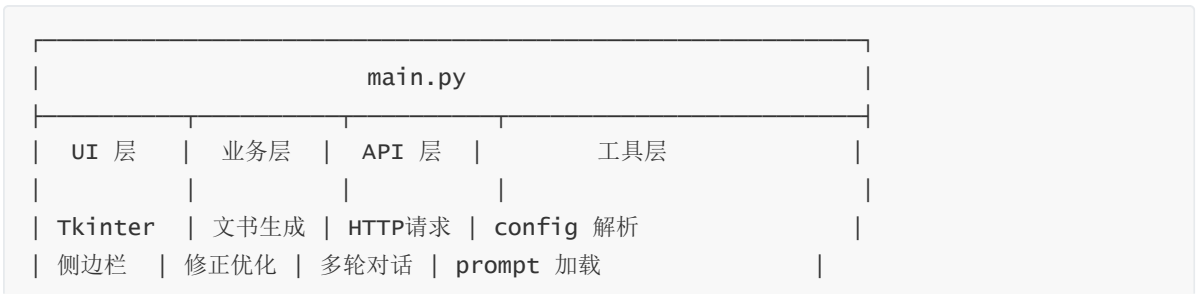
- 1. **资料收集**：整理大量真实法律文书档案作为知识源（包括司法实践模板、标准文书范本、优秀法律文书等）
- 2. **增强检索**：针对每一种法律文书（如民事起诉状），使用 RAG 技术从知识源中检索最相关的参考资料
- 3. **逐篇生成**：对检索到的资料进行归纳整理，一篇一篇地为每种文书类型生成专业级系统提示词
- 4. **人工校验**：对生成的提示词进行人工审核，确保用语规范、格式准确、法律表述严谨
- 5. **模板入库**：最终将 17+ 份提示词以 `.txt` 文件形式存入 `prompt/` 目录，作为系统的核心知识库

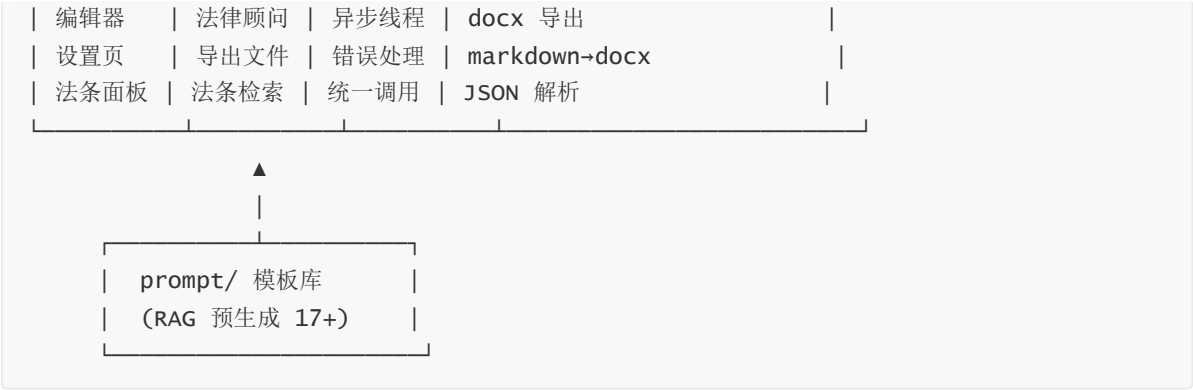
双层架构设计（在线运行）



整体架构

项目采用单文件架构（`main.py`），基于 Tkinter 构建 GUI 界面，通过 HTTP 请求调用兼容 OpenAI 格式的 DeepSeek 大模型 API 实现核心功能。





核心模块说明

模块/类	说明
<code>load_config()</code> / <code>_save_settings()</code>	读写 <code>config.ini</code> , 管理 DeepSeek API 配置
<code>load_prompt()</code> / <code>_find_prompt_file()</code>	按文书名称查找并加载对应 RAG 生成的提示词模板 , 优先读取用户目录
<code>_do_api_call()</code>	统一 API 调用核心函数, 处理请求发送与异常捕获
<code>call_api()</code>	单轮 API 调用, 将 RAG 提示词 作为 system 消息 与用户材料组合后发送给 DeepSeek
<code>call_api_chat()</code>	多轮对话 API 调用, 维护完整对话历史, 用于智能法律顾问功能
<code>LegalDocumentApp</code>	主应用类, 包含全部 UI 构建与业务逻辑
<code>HoverButton</code>	自定义侧边栏按钮组件, 支持悬停高亮与激活态
<code>CollapsibleSection</code>	可折叠分类组件, 用于文书类型树形导航
<code>FlatButton</code>	自定义扁平化按钮组件
<code>_markdown_to_docx()</code>	Markdown → HTML → docx 转换, 处理标题/表格/列表/加粗等格式, 中文字体自动适配

工作流程

- 文书生成**: 用户选择文书类型 → 加载 **RAG 预生成的对应提示词模板** → 输入案件材料 → 调用 DeepSeek API 生成 → 展示结果
- 修正优化**: 读取已生成/输入的文书内容 → 加载「修改优化」**RAG 提示词** → 调用 API 优化 → 展示结果
- 智能法律顾问**: 加载 **RAG 生成的法律顾问提示词** → 用户自由提问 → 多轮对话持续交互
- 类案推送与法条关联**: 文书生成后自动触发 → 异步调用 API 检索关联法条、相似案例与风险提示 → 右侧可折叠面板实时展示, 支持手动重新检索
- 导出文件**: 将 Markdown 格式的生成结果转换为 Word 文档, 设置中文字体与页边距

提示词模板机制

每种文书在 `prompt/` 目录下有对应的 `.txt` 文件作为系统提示词（如 `民事起诉状.txt`）。这些模板均由 **RAG 技术从法律文书资料档案中逐篇增强检索后生成**，包含明确的角色定位、任务要求、格式规范与风险提示。模板查找顺序：

1. 程序运行目录下的 `prompt/` 文件夹（用户自定义，优先）
2. PyInstaller 打包内置的 `prompt/` 文件夹（内置默认，**RAG 预生成**）

用户可在「系统设置 → 提示词模板管理」中直接编辑模板，修改保存到运行目录下，无需重新打包。

项目结构

```
智能法律文书系统/
├─ main.py                # 主程序（GUI + 业务逻辑）
├─ main.spec              # PyInstaller 打包配置
├─ config.ini             # API 配置文件（需自行创建/配置）
├─ requirements.txt       # Python 依赖
├─ my.ico                 # 应用图标
├─ LICENSE                # 许可证
├─ README.md              # 项目说明
├─ prompt/                # 提示词模板目录
│   └─ 民事起诉状.txt
│   └─ 答辩状.txt
│   └─ 保全申请书.txt
│   └─ 民事上诉状.txt
│   └─ 民事再审申请书.txt
│   └─ 行政起诉状.txt
│   └─ 行政上诉状.txt
│   └─ 行政再审申请书.txt
│   └─ 举报信.txt
│   └─ 检举信（纪委监委）.txt
│   └─ 控告书（纪委监委）.txt
│   └─ 修改优化.txt
│   └─ 智能法律顾问.txt
├─ dist/                  # 打包输出目录
│   └─ 智能法律文书服务系统.exe
│   └─ config.ini
└─ build/                 # PyInstaller 构建临时目录
```

版本日志

v1.1.0 ----2026.6.15

- 🌟 新增「类案推送与法条关联」功能
 - 文书生成后自动检索关联法律条文、相似参考案例与诉讼风险提示
 - 右侧可折叠面板实时展示，支持展开/收起切换
 - 面板右上角 × 按钮与工具栏「📁 法条」按钮双重控制
 - 状态栏显示具体检索数量（如：法条3条 · 类案2件 · 风险1项）
 - 支持手动重新检索

- 🐛 修复 `FlatButton.configure_state` 按钮禁用后恢复时颜色错误 (硬编码蓝色覆盖原始颜色)
- 🐛 修复 `call_api` / `call_api_chat` 大量重复代码及 `resp` 变量未定义导致的潜在异常
- 🐛 修复右侧面板鼠标滚轮 `bind_all` 全局绑定导致编辑器区域滚动冲突
- 🐛 修复 API 超时时间硬编码问题, 提取为 `API_TIMEOUT` 常量
- 🧠 优化窗口关闭时增加确认对话框, 防止误操作
- 🧠 优化右侧面板布局, 固定宽度 320px 不挤压左侧编辑器

v1.0.0 ----2026.6.14

- 初始版本发布
- 支持 17+ 种法律文书类型的智能生成
- 实现文书修正优化功能
- 实现智能法律顾问多轮对话
- 支持 Word (.docx) 文档导出
- 支持提示词模板管理与编辑
- 支持 API 配置管理
- PyInstaller 打包为独立 exe

许可证

本项目采用 Prosperity Public License 2.0.0 许可证, 详见 LICENSE 文件。

核心限制: 本软件仅供个人学习、研究、非商业用途。任何形式的商业使用 (包括企业内部使用、盈利服务、销售分发等) 均需获得书面授权。商业授权请联系: 430615396@qq.com